Documento de requerimientos y especificaciones técnicas

PROGRAMACIÓN IV · TP N.º 2 · RED SOCIAL · Guía de implementación para Codex

Versión 1.0 | Basado en el enunciado de 9 páginas, «TP #2 - Red social» (UTN Avellaneda, 2026).

1. Objetivo y reglas de implementación

Construir una red social web donde usuarios registrados publiquen contenido, indiquen «Me gusta», comenten, consulten perfiles y, según su rol, administren usuarios y publicaciones. El proyecto se divide obligatoriamente en frontend Angular y backend NestJS, con MongoDB y autenticación JWT.

- Desarrollar de manera incremental. SOLO implementar la lógica que corresponde al sprint activo. No adelantar funcionalidades de sprints posteriores.
- Cuando la consigna pida crear pantallas, componentes o módulos sin funcionalidad todavía, implementar un MOCKUP visual navegable: datos ficticios locales, controles ilustrativos y sin llamadas HTTP reales. Etiquetar internamente el estado como pendiente de implementación.
- Conservar intactos componentes, rutas, entidades y estilos ya entregados. Agregar funcionalidades sprint por sprint sin rehacer innecesariamente el proyecto.
- Priorizar los patrones y herramientas vistos en clase: Angular (componentes, routing, formularios reactivos, servicios, guards/interceptors cuando corresponda, pipes, directivas); NestJS (módulos, controladores, servicios, DTO/validación, guards); MongoDB. No añadir frameworks alternativos como React, Next.js, Express independiente, Firebase Auth, Supabase Auth o bases SQL.
- Usar modales en lugar de alert() y una UI consistente, responsive y no basada en HTML plano.
- Las decisiones técnicas no definidas literalmente por la cátedra se documentarán como propuestas, no como requisitos formales.

2. Alcance, arquitectura y convenciones

Capa	Requerimiento	Criterio
Front	Angular + TypeScript + CSS	Proyecto separado del backend, rutas y componentes
Back	NestJS + TypeScript	API propia, organizada en módulos
Persistencia	MongoDB	Documentos/colecciones e índices de unicidad
Sesión	JWT	Expiración 15 minutos a partir del Sprint 3
Despliegue	Frontend en hosting	Exigido en Sprint 1; backend accesible cuando proceda
PWA	Angular PWA	Implementación en Sprint 4

Estructura sugerida (no obligatoria):
frontend/src/app/{pages,components,services,guards,interceptors,pipes,directives};
backend/src/{auth,users,posts,common}; carpetas de DTO, schemas, controllers y services dentro de cada módulo.
Guardar configuración sensible en variables de entorno; nunca subir secretos al repositorio.

3. Matriz de entregas y mockups

Sprint	Frontend funcional	Mockup / base sin lógica	Backend funcional
1	Registro y login con validaciones; navegación; deploy	Publicaciones y Mi Perfil; módulos NestJS de publicaciones y usuarios	Registro/login persistidos en MongoDB e imagen de perfil
2	Feed, orden, paginación, me gusta, eliminación propia; Mi Perfil	Pantalla de detalle de post / comentarios, si se prepara anticipadamente: solo vista	Alta, listado, baja lógica de posts y likes
3	Detalle, comentarios, edición; sesión, loading, refresh	—	Comentarios, autorización y renovación JWT
4	Administración, gráficos, PWA, pipes y directivas	—	Usuarios/estadísticas con permisos de administrador
5 (recuperatorio)	Perfiles ajenos, scroll infinito y métricas adicionales	—	Soporte para nuevas métricas y visitas
6 (final)	Guardados, compartidos, dos ordenamientos	—	Persistencia y API para guardados/compartidos

4. Requisitos transversales

- Interfaz: diseño visual coherente, favicon propio, menús y navegación clara, formularios con validaciones y estados de carga/vacío/error. Confirmaciones y avisos a través de modales.
- Permisos: distinguir rol «usuario» y «administrador»; el rol predeterminado en registro público es «usuario». Nunca confiar en un rol enviado por el frontend para autorizar acciones.
- HTTP: devolver 201 al crear, 200 para consultas exitosas, 400 ante datos inválidos, 401 por credenciales/token inválidos, 403 por falta de permisos, 404 si no existe recurso, 409 en conflictos de unicidad, según corresponda. Nunca responder 200 con error.
- Privacidad: nunca enviar hashes de contraseña en respuestas, pese a que el enunciado indique devolver datos del usuario. Entregar todos los datos de perfil pertinentes y el token cuando corresponda.
- Imágenes: aceptar imagen opcional cuando el requisito lo establece, validar el archivo, almacenarlo y guardar una URL accesible en MongoDB.
- Integridad: baja lógica de posts/usuarios; los eliminados no deben aparecer en listados públicos ni quedar disponibles por URL.
- Accesibilidad y UX: etiquetas en formularios, feedback de error específico, acciones deshabilitadas si no se cumplen condiciones, diseño adaptable a móvil.

5. Sprint 1 — Inicio, autenticación y maquetas

5.1 Frontend — obligatorio

- Crear el proyecto Angular, configurar rutas y desplegar el frontend en un hosting. Crear y aplicar favicon propio.

- Rutas/pantallas: /registro, /login, /publicaciones y /mi-perfil. En esta primera entrega la navegación entre componentes será libre («sin límites de accesibilidad»), tal como expresa la consigna.
- Login: identificador por correo o nombre de usuario (elegir un criterio y mantenerlo consistente), contraseña, campos requeridos, mensajes de error y formato. Contraseña: mínimo 8 caracteres, una mayúscula y un número.
- Registro: nombre, apellido, correo, nombre de usuario, contraseña, repetir contraseña, fecha de nacimiento, descripción breve y selector de imagen de perfil tipo file. Validar coincidencia de contraseñas y campos obligatorios.
- Pantalla Publicaciones — MOCKUP: encabezado, menú, listado de tarjetas ficticias con título, texto, imagen opcional, autor, fecha, contador de likes y comentarios, área de acciones visuales. No habilitar API de posts, likes ni comentarios todavía.
- Pantalla Mi Perfil — MOCKUP: avatar, nombre completo, usuario, correo, nacimiento, biografía y 3 publicaciones de muestra. La edición real del perfil no está detallada en los sprints; crear un botón/estructura visual sin fingir persistencia y señalar el punto a confirmar con la cátedra.

5.2 Backend — obligatorio

- Crear NestJS, conectar MongoDB y definir módulos Autenticación, Publicaciones y Usuarios; en este sprint Publicaciones y Usuarios pueden quedar como módulos estructurales sin endpoints funcionales de gestión.
- POST /auth/register (ruta sugerida): validar los campos, comprobar unicidad de correo y nombre de usuario, almacenar hash seguro de contraseña, guardar imagen y URL de avatar, asignar rol «usuario» por defecto.
- POST /auth/login: recibir identificador y contraseña, comprobar credenciales con comparación de hash; devolver datos del usuario (sin contraseña).
- El enunciado incorpora la generación explícita de JWT recién en Sprint 3: preparar la estructura ahora, sin obligar a implementar renovación o control de expiración en Sprint 1.

5.3 Criterios de aceptación

- Se puede recorrer cada pantalla y el frontend está desplegado.
- Los formularios muestran errores legibles; contraseña inválida o contraseñas distintas impiden continuar.
- Un registro válido crea el usuario en MongoDB con perfil «usuario» y URL de avatar si se adjuntó.
- Un segundo registro con correo o nombre de usuario repetido se rechaza; el login comprueba credenciales.
- Publicaciones y Mi Perfil se ven bien, pero NO aparentan tener persistencia o interacciones de sprints posteriores.

6. Sprint 2 — Publicaciones, likes y Mi Perfil

6.1 Frontend

- Feed: recuperar publicaciones reales de la API, ordenadas por fecha de más nueva a más vieja; selector para cambiar a cantidad de «Me gusta».
- Paginación mediante offset y limit, trayendo cantidades acotadas. Renderizar cada publicación mediante un componente reutilizable.
- Permitir dar/quitar un único like propio y eliminar publicaciones propias con confirmación modal.
- Ofrecer formulario de creación de publicación (título, mensaje, imagen opcional) para consumir el POST que exige el backend.

- Mi Perfil: mostrar los datos reales del usuario, foto y sus 3 publicaciones más recientes junto con comentarios disponibles; si el controlador de comentarios aún no existe, usar una maqueta de comentarios claramente temporal hasta Sprint 3, sin inventar un endpoint.

6.2 Backend

- POST /posts: alta de publicación asociada a un usuario, título, descripción e imagen opcional; almacenar imagen y URL.
- GET /posts?sort=date|likes&offset=0&limit=10&userId=...: listado por fecha o likes, opcionalmente filtrado por usuario y paginado; excluir bajas lógicas.
- DELETE /posts/:id: baja lógica permitida solo para autor o administrador.
- POST /posts/:id/likes: crear like único de un usuario en ese post.
- DELETE /posts/:id/likes: quitar like únicamente cuando el usuario ya lo había otorgado.
- La identificación del usuario deberá provenir de una identidad validada; como el JWT se exige formalmente en Sprint 3, coordinar la implementación progresiva sin dejar mecanismos de suplantación expuestos en un despliegue público.

6.3 Criterios de aceptación

- Se puede crear un post, verlo en el feed, filtrar por autor, paginar y cambiar orden.
- Un usuario no puede incrementar dos veces el contador de likes del mismo post.
- La eliminación oculta post y comentarios asociados de las consultas públicas; otro usuario no puede borrar publicaciones ajenas.
- Mi Perfil muestra los 3 posts reales más recientes; comentarios con datos reales solo cuando ya exista su API.

7. Sprint 3 — Comentarios y seguridad de sesión

7.1 Frontend

- Vista /publicaciones/:id: mostrar post ampliado y comentarios ordenados del más reciente al más antiguo.
- Formulario para agregar comentario. Cargar un primer lote limitado y un botón «Cargar más» que agregue resultados sin borrar los previos.
- Permitir que solo el autor edite un comentario; mostrar etiqueta «Editado» cuando modificado=true.
- Login/registro almacenan token JWT según estrategia elegida (localStorage o cookies apropiadas).
- Vista inicial /cargando con spinner; POST /auth/authorize verifica token y dirige al feed o login según validez.
- A los 10 minutos desde el inicio de sesión, mostrar modal «Quedan 5 minutos, ¿querés extender la sesión?»; al confirmar, POST /auth/refresh y reiniciar control según el nuevo token.
- Si cualquier solicitud protegida devuelve 401, limpiar estado de sesión y llevar a /login; evitar loops de redirección.

7.2 Backend

- POST /posts/:id/comments: guardar comentario asociado a post y autor, con fecha.
- GET /posts/:id/comments?offset=0&limit=10: listar por fecha descendente y paginar.
- PUT /posts/:id/comments/:commentId: editar solo comentario del autor y marcar modificado=true.
- POST /auth/authorize: validar firma y vencimiento del JWT; retornar 401 en error o datos públicos del usuario si es válido.

- POST /auth/refresh: recibir token no vencido, validarlo y crear JWT nuevo con la misma payload y otros 15 minutos de validez.
- POST login y registro: generar JWT con identidad (id/correo/nombre de usuario) y rol. Vencimiento: exactamente 15 minutos.

Nota técnica: el enunciado exige refrescar un token todavía válido. No confundir este flujo con refresh tokens de larga duración; no agregar uno adicional sin autorización docente.

8. Sprint 4 — Administración, estadísticas y PWA

8.1 Frontend

- Mostrar opción de eliminar cualquier post a quienes tengan rol administrador, tanto en feed como en detalle. Al deshabilitarlo no debe poder verse el post ni sus comentarios.
- /dashboard/usuarios: página exclusiva de administradores; tabla de usuarios, alta con campos de registro y radios «usuario»/«administrador», botones habilitar y deshabilitar (baja y alta lógica).
- Usuario deshabilitado: no debe iniciar sesión y debe recibir mensaje claro de falta de autorización.
- /dashboard/estadisticas: gráficos con filtros por rango de fechas: cantidad de publicaciones por usuario, cantidad total de comentarios y cantidad de comentarios por publicación. Variar los tipos de gráfico (barras, líneas, torta, etc.).
- Convertir a PWA (manifest y service worker de Angular) y verificar que sea instalable bajo HTTPS en despliegue.
- Crear tres pipes propios, por ejemplo nombreCompleto, tiempoRelativo y textoResumido; crear tres directivas propias, por ejemplo resaltarAutor, rolVisible y focoInicial. Son propuestas: usar transformaciones DOM y usos visibles reales, no solo archivos vacíos.

8.2 Backend

- GET /users: listado accesible solo a administradores.
- POST /users: alta administrativa pudiendo definir rol.
- DELETE /users/:id: deshabilitación lógica; POST /users/:id/enable: rehabilitación lógica.
- GET /stats/posts-by-user, /stats/comments-over-time y /stats/comments-by-post: retornar series agregadas, con from/to y acceso admin.
- En todas las rutas administrativas, validar JWT y rol desde el servidor, no solo ocultar botones en Angular.
- Bloquear también accesos con token previamente emitido si el usuario ahora está deshabilitado: consultar su estado en operaciones protegidas.

8.3 Criterios de aceptación

- Usuario común no accede al dashboard ni ejecuta operaciones de administración.
- Administrador da de baja y rehabilita usuarios; sus publicaciones administradas desaparecen del feed.
- Los 3 gráficos cambian correctamente al variar fechas; son de tipos diferentes.
- PWA instalada/verificada y 3 pipes + 3 directivas propias integrados en pantallas reales.

9. Sprint 5 — Recuperatorio (separado del alcance ordinario)

- Perfiles de otros usuarios: clic en avatar/nombre para consultar información y sus tres últimas publicaciones; conservar facultad de moderación para administrador.
- Reemplazar paginación visible del feed por scroll infinito, sin perder los posts ya cargados.
- Agregar métricas: cantidad de logins por usuario; visitas a un perfil hechas por terceros (excluir visitas del propio dueño); likes otorgados por día.
- Backend: registrar eventos mínimos que permitan calcular las métricas y exponer estadísticas protegidas; definir tratamiento de visitas repetidas con el docente si fuera relevante.

10. Sprint 6 — Fecha de final (alcance adicional)

- En detalle de publicación: guardar y quitar de guardados con estado visible.
- /guardados: listar e interactuar con las publicaciones guardadas por el usuario autenticado.
- Compartir post desde detalle mediante selector de destinatario; guardar identidad de remitente, destinatario y post.
- /compartidos: mostrar posts recibidos e identidad de quien los compartió.
- Ampliar los ordenamientos por número de guardados y número de compartidos.
- Backend: endpoints POST para guardar, quitar de guardados y compartir; GET de compartidos recibidos; cálculos de ordenamientos. Derivar usuario actor del token, nunca del body.

11. Modelo de datos propuesto (MongoDB)

Colección	Campos principales	Relaciones / reglas
users	_id, nombre, apellido, email, username, passwordHash, fechaNacimiento, bio, avatarUrl, role, active, createdAt	email y username únicos; role usuario/admin
posts	_id, authorId, title, description, imageUrl?, createdAt, deletedAt?, likesCount	Baja lógica; autor referenciado
likes	_id, postId, userId, createdAt	Índice único {postId,userId}
comments	_id, postId, authorId, message, createdAt, updatedAt?, modificado	Autor y post referenciados
savedPosts [Sprint 6]	_id, postId, userId, createdAt	Índice único por usuario/post
shares [Sprint 6]	_id, postId, senderId, recipientId, createdAt	Registro de compartidos
loginEvents [Sprint 5]	_id, userId, createdAt	Conteo de inicios de sesión
profileVisits [Sprint 5]	_id, visitorId, profileId, createdAt	Excluir visitas del propietario

Los nombres exactos de endpoints, campos y colecciones son propuestas técnicas para que Codex implemente de forma consistente. La cátedra exige comportamientos, no esas rutas literales. El proyecto debe incorporar estas colecciones solo desde el sprint en que resultan necesarias.

12. Pantallas y especificación de mockups

Pantalla	Composición visual	Momento de activación

Login / Registro	Tarjeta centrada, logo, campos, mensajes y acceso entre páginas	Formulario + API en Sprint 1
Publicaciones	Header, feed, selector de orden, tarjetas, acciones, vacío	Mockup S1 → funcional S2
Mi Perfil	Avatar, bio, datos, bloque de 3 publicaciones, botón editar	Mockup S1 → datos S2
Detalle publicación	Tarjeta ampliada, autor, imagen, comentarios, editor	Funcional Sprint 3
Dashboard usuarios	Tabla, búsqueda, botones de estado, modal de creación	Sprint 4
Dashboard estadísticas	Filtros de fechas, tarjetas KPI y gráficos variados	Sprint 4
Perfil ajeno	Datos públicos, 3 publicaciones	Sprint 5 recuperatorio
Guardados / Compartidos	Listados, indicadores de estado, autor/remitente	Sprint 6 final

Norma para mockups: usar un array local de ejemplos tipados dentro del componente o archivo mocks; no conectar al backend, no escribir en MongoDB, no hacer botones que pretendan haber persistido acciones. Acciones futuras pueden aparecer visualmente deshabilitadas con texto «Disponible en próximo sprint». Sustituir los mocks por servicios reales recién en la entrega correspondiente.

13. Contratos API sugeridos y reglas de prueba

Método	Ruta propuesta	Se incorpora
POST	/auth/register · /auth/login	Sprint 1
POST/GET/DELETE	/posts · /posts/:id	Sprint 2
POST/DELETE	/posts/:id/likes	Sprint 2
POST/GET/PUT	/posts/:id/comments · /posts/:id/comments/:commentId	Sprint 3
POST	/auth/authorize · /auth/refresh	Sprint 3
GET/POST/DELETE	/users · /users/:id	Sprint 4
POST	/users/:id/enable	Sprint 4
GET	/stats/*	Sprint 4 (ampliaciones S5)
POST/GET	/posts/:id/save · /posts/:id/unsave · /posts/:id/share · /shares/received	Sprint 6

- Usar DTO y validación del backend aun cuando Angular valide; errores coherentes entre ambos.
- Probar al menos: registro válido, duplicados, contraseña inválida, credenciales incorrectas, acceso sin token y expirado, usuario deshabilitado, rol usuario intentando ruta admin.
- Probar posts: create, filtrado/paginación, likes duplicados, baja lógica por autor/admin, post eliminado consultado por URL.
- Probar comentarios: autor que edita, tercero que intenta editar, paginación por «Cargar más» y etiqueta editado.

- En sprint 4 y siguientes probar estadísticas por rango y restricción por rol.
- Verificar comportamiento responsive, modal de expiración, favicon y PWA instalada cuando correspondan.

14. Guía de trabajo para Codex: instrucciones de ejecución

Prompt base que debe seguir Codex al implementar cada entrega:

1. Antes de codificar, revisar el repositorio y determinar qué sprint está implementado. Respetar sus convenciones y conservar lo ya funcional.

2. Implementar ÚNICAMENTE los requisitos del sprint solicitado en este documento. No adelantar funciones de otros sprints aunque se creen sus componentes.

3. Si una pantalla aún no tiene funcionalidad pedida, realizar mockup pulido con datos de muestra explícitos y navegación. Si un módulo del backend solo debe existir, dejar estructura válida y compilable sin fingir endpoints.

4. Mantener Angular + NestJS + MongoDB. Preferir las técnicas vistas en la materia; ante una dependencia no vista, elegir solución simple o dejar decisión documentada.

5. Separar presentación, servicios HTTP, validaciones, reglas de negocio, persistencia y autorización. No guardar contraseñas en texto plano.

6. Al finalizar, ejecutar build/lint/tests disponibles y reportar: archivos cambiados, rutas nuevas, prueba manual de aceptación, configuraciones requeridas, decisiones propias y tareas pendientes por sprint.

7. Nunca marcar como funcional una maqueta. No eliminar o modificar requerimientos de la consigna.

15. Puntos a confirmar con el docente

- La consigna general pide que cada usuario pueda editar su perfil, pero ningún sprint describe un endpoint/pantalla concreta para guardar cambios. Preparar la maqueta, y confirmar en qué sprint se evalúa la edición real.
- El Sprint 2 exige mostrar comentarios en las tres publicaciones de Mi Perfil, mientras que el controlador de comentarios recién está definido en Sprint 3. Implementar la visualización sin anticipar el módulo de comentarios y confirmar el criterio de corrección.
- La consigna no indica plataforma de almacenamiento de archivos ni librería específica para gráficos. Elegir una opción sencilla compatible con clase y documentarla antes de incorporarla.
- En Sprint 1 habla de comprobar credenciales y «encriptar» contraseñas, mientras JWT se incorpora explícitamente en Sprint 3. Usar hash con sal y comparación segura, y mantener la progresión por sprint.

FUENTE: «PROGRAMACIÓN IV - TP #2 - Completo - 2026 C1», páginas 1–9. Requisitos del docente separados de propuestas de implementación donde corresponde.